

ASK THE SENSORS

Answering questions on sensor data, with evidence

CS60055 Ubiquitous Computing

Hackathon Challenge 1

Presented by

Abhinav Kumar Singh

22CS30005

Harsh Chattar

22CS30028

The problem

Scenario. An older woman lives alone. She has a phone that carries motion sensors. Her son and her physiotherapist want to know how she is doing, based on the data those sensors record. We need to build a tool that answers their questions in **plain English** and also **provides the evidence** behind each answer.

What the tool must produce — the example given in the challenge brief:

Query: "How long was the user walking?"

Answer: 700 seconds

Activity/Event: Walking

Evidence:

Timestamp(s): 905 to 1420, 2110 to 2295 (seconds from start)

Sensor Modality: Accelerometer, Gyroscope

Sensor Channel(s): All

Explanation: Walking was detected in two separate intervals, of 515 and 185 seconds, which sum to 700 seconds.

Not just *"700 seconds"* — but **which stretches of signal**, from **which sensors**, and **why** they read as walking.

Functional requirements: four question tiers, one system

The evaluation mixes all four tiers, without saying which is which.

Tier	Example question	What must be produced
1. Identification	"What activity is she performing?" "Is she running?"	The activity, or a yes/no verdict
2. Temporal & quantitative	"How long was she walking?" "How many times?"	Numbers computed across the <i>whole</i> recording, with intervals
3. Evidence grounding	"Did she begin running, and when?"	Answer <i>plus</i> timestamps, sensor modality, channels, and the reasoning
4. Open-world reasoning	"Was she resting for a long time?" "Was she fidgeting?"	A verdict on behaviour <i>outside</i> the seven trained labels, argued from the signal

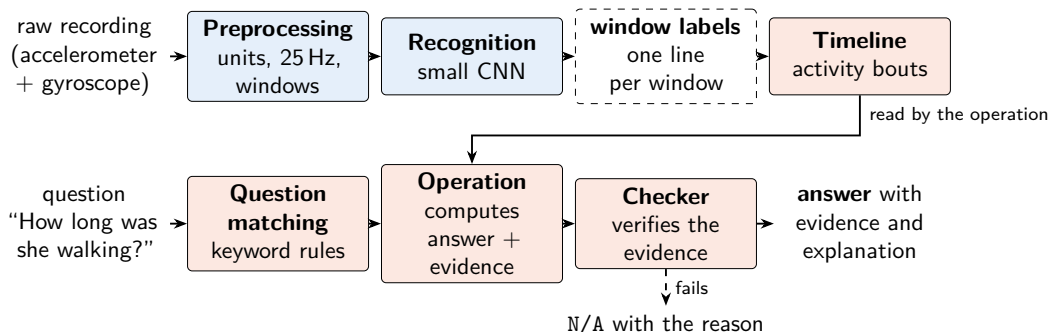
The seven activities we recognise

Lying, sitting, standing in place, standing and moving, walking, running, bicycling.

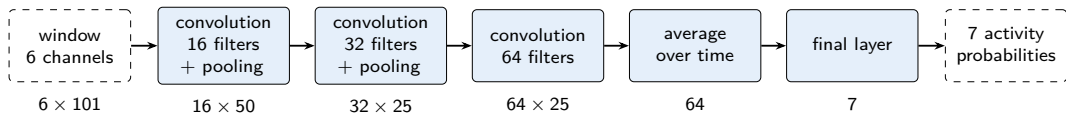
One fixed output format, every tier

Answer, activity/event, evidence (timestamps, modality, channels), explanation. N/A is the only permitted blank.

Architecture: two halves, one agreed file



Recognition: a deliberately small network

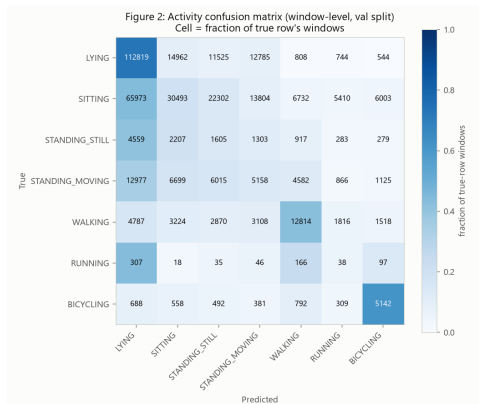


Input. A 4-second window, a new one every 2 seconds: 6 channels $\times$ 101 samples at 25 Hz.

Why so small? **9,975 parameters, 48 KB** — it runs on a phone and compresses cleanly (slide 9).

Trained on 36 people, validated on 12 others; nobody appears in both. Rare activities are weighted up (running is 0.6% of windows).

Result. macro-F1 **0.289**, against 0.079 (always guess the commonest activity) and 0.225 (logistic regression).



The rule that shapes the whole design

A language model **never** produces a number, an interval or a verdict. Every quantity is computed in ordinary code from the timeline, so it can be recomputed and checked. The small language model we tested chooses only *which operation runs*; it never sees the signal.

1. Match the question to one of eight operations:

Operation	Example
Identify	"What was she doing at 340 s?"
Verify	"Is she running?"
Duration	"How long was she walking?"
Count	"How many times did she walk?"
Compare	"More time walking or running?"
Find start	"When did she begin running?"
Show evidence	"Cite the evidence for walking."
Open-ended	"Was she resting for a long time?"

2. Compute the answer and the intervals behind it, quoting the measured signal in the explanation.

3. Check — before anything is shown:

- every cited interval **exists** in the timeline;
- the printed times **match** those intervals;
- tiers that need evidence **have** it;
- durations and counts **add up again** from the intervals.

A failing answer is withheld and replaced by N/A with the reason. There is no bypass.

Open-world questions rest on measured stillness, not on labels.

Real output: four questions, one recording

Tier 1 — identification

Query: "What activity ... at 340 seconds?"

Answer: Sitting

Activity/Event: Sitting

Evidence:

Timestamp(s): 0 to 600 (seconds from start)

Modality: Accelerometer, Gyroscope

Channel(s): All

Explanation: 340 s falls inside 0-600 s,
classified as sitting (confidence 0.85);
100 windows, mean 9.80 m/s², std 0.08.

Tier 2 — duration

Query: "How long was the user walking?"

Answer: 720 seconds

Evidence:

Timestamp(s): 600 to 1080, 1320 to 1560

Explanation: Walking was detected in 2
interval(s) of 480, 240 s, summing to
720 s; cadence near 1.90 Hz in 100% of
windows.

Verbatim system output; explanations trimmed to fit.

Tier 3 — evidence grounding

Query: "Did the user begin walking, when?"

Answer: Yes, walking began at 600 s

Activity/Event: Onset of walking

Evidence:

Timestamp(s): 600 to 1080

Explanation: The first walking interval
runs 600-1080 s, following sitting; across
the transition accelerometer std goes
0.08 -> 1.55 m/s² and gyroscope energy
0.124 -> 1.45.

Tier 4 — refusing to guess

Query: "Was the user driving?"

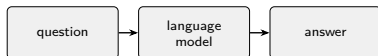
Answer: N/A

Evidence: N/A

Explanation: This question does not match
any reasoning pattern the system supports,
and answering it would mean guessing beyond
the recorded evidence.

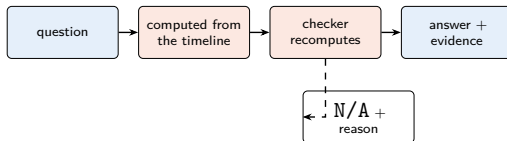
What we did differently (1): grounding, and a timeline that fits the data

The usual way

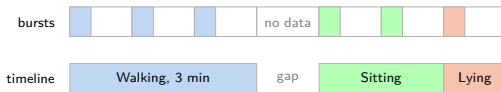


nothing checks it

Ours



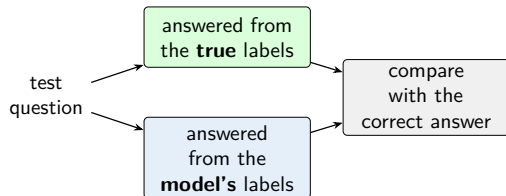
Grounding by construction. Every number and interval is recomputed before release; anything that fails is withheld. **An ungrounded answer is structurally impossible**, not just unlikely.



each burst stands for its whole minute

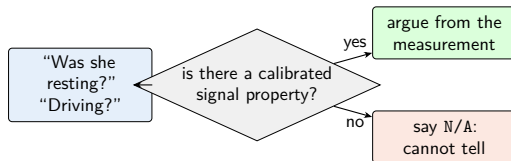
A timeline that fits the data. The phone records about 20 s per labelled minute. Summing windows would report **a third of reality**; bridging clock drift turned **3,151 fragments into 325 real bouts**.

What we did differently (2): tracing errors, and untrained behaviours



Two tracks, identical code. Right with true labels but wrong with the model's $\Rightarrow$ a **recognition** error; wrong either way $\Rightarrow$ an **answering** error. This is *how* we can say **all 19 misses are recognition errors**.

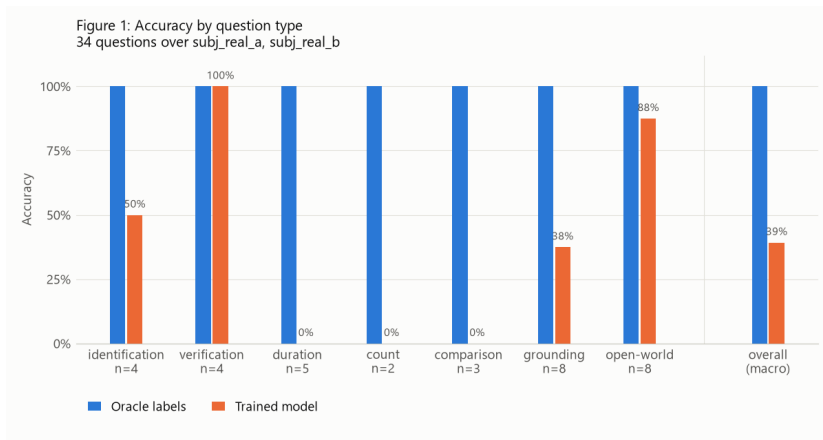
We also built the small-language-model path and **measured it losing** to keyword rules — then reported that, rather than shipping the fancier option.



Untrained behaviours, still answered honestly.

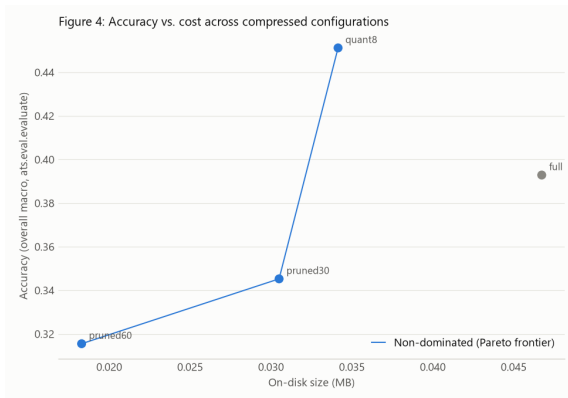
Stillness is calibrated from data, with a per-recording gyroscope-bias correction — and where no signature exists, the system **declines rather than guesses**.

Results: the answering layer is not the bottleneck



- **True labels:** 87 of 87 questions correct.
- **Model's labels:** 15 of 34 (39%) — and all 19 misses are recognition errors, none from the answering layer.
- **Durations and counts collapse:** they accumulate per-minute errors over tens of hours.
- **Stress test:** mislabel 10% of minutes and the score still holds at 84%.

Efficiency: small to begin with, smaller after compression



- On the target laptop: 9,975 parameters, 0.047 MB, **0.40 ms** per window, **67 ms** per question.
- **Three compressed versions, no retraining:** 8-bit weights, 30%/60% of filters removed.
- The 8-bit model is **27% smaller at no accuracy cost**; compression buys *space*, not speed.
- **Robustness:** dropping half the raw samples costs almost nothing (0.393 $\rightarrow$ 0.375).

Limitations, and how we used AI tools

Limitations

- **Recognition caps everything.** macro-F1 0.289 limits the system to 15 of 34 on real users, even though the answering layer made no errors.
- **Optimistic numbers.** Both real test users are in the training group; held-out users would do worse.
- **Small, self-written question sets** (87 questions), and a router tuned against phrasings we wrote ourselves.
- **Stillness is our only calibrated property;** other behaviours rely on labels or decline to answer.
- **Energy is an estimate**, not a meter reading, and only one kind of input damage was tested.

Use of AI tools

Inside the system. Qwen2.5-0.5B as an optional question parser, and Claude Haiku 4.5 to score explanations during evaluation. Neither ever produces an answer the system gives.

As development assistants. Claude Code was used to help write and debug code and tests, to explain concepts and documentation, and to help draft the report.

We directed the work, made the design decisions, reviewed what the tools produced, and **checked every reported number by running our own code.**

Abhinav Kumar Singh (22CS30005)

The signal half.

- Data download, parsing and per-phone unit detection
- Resampling to 25 Hz, gap policy, windowing, subject splits
- The recognition model and its confusion analysis
- Cost measurement, model compression, robustness testing

Harsh Chatter (22CS30028)

The answering half.

- The activity timeline and evidence attribution
- Question matching and the eight operations
- Signal-based answers for open-ended questions
- The checker that blocks unsupported answers
- The whole evaluation: test questions, scoring, error tracing

Both halves meet at one frozen file format, which is why each could be built and tested independently.

THANK YOU

Questions?

Abhinav Kumar Singh

22CS30005

Harsh Chattar

22CS30028

github.com/code-Harsh247/AskTheSensors